

Software engineering fundamentals that work with humans also work exceptionally well with AI — the key is understanding LLM constraints and applying proven development practices.

[

LLM Constraints: Smart Zone vs Dumb Zone

- LLMs have a ‘smart zone’ (early context) and ‘dumb zone’ (after 100K tokens). Attention relationships scale quadratically with each token, degrading performance as context grows.
- Size tasks to stay in the smart zone. Break large tasks into small chunks rather than letting context bloat into the dumb zone.
- Multi-phase plans are just loops. Use phase N approach: maintain a plan in background, loop until complete, keeping each iteration in smart zone.
- LLMs are like the guy from Memento — they reset to base state. Every session goes: system prompt → exploration → implementation → testing. Clearing context returns to system prompt.
- Avoid compacting. Prefer clearing context over compacting history. Compacting creates ‘sediment’ that degrades quality. Clean resets maintain consistent state.

Don't bite off more than you can chew. Keep your tasks small so the AI doesn't freak out and go into the dumb zone.

From Alignment to Action: PRD and Issues

- Create two documents: destination (PRD) and journey (implementation plan). PRD describes what you're building, plan describes how.
- Use a Kanban board with blocking relationships. Break work into parallelizable issues that can be worked on independently.
- Don't over-optimize the PRD. The value is in alignment (grilling session) and QA, not perfecting the document.
- PRDs can cause doc rot if kept in repo long-term. Code changes faster than documentation. Consider closing/archiving after implementation.
- Think in terms of human-in-the-loop tasks (planning, alignment, QA) vs AFK tasks (implementation, testing).

Code Architecture: Deep vs Shallow Modules

- Shallow modules (many small files with complex dependencies) are hard for AI to navigate and test. Deep modules (simple interface, rich functionality) are ideal.
- Deep modules have clear test boundaries. Wrap entire module in test boundary rather than testing each tiny function.
- Design module interfaces yourself, delegate implementation to AI. Treat modules as gray boxes — know the shape and behavior, not every internal detail.
- This preserves your understanding of the codebase while moving fast. You maintain the big picture without drowning in implementation details.
- Use ‘improve code base architecture’ skill to scan repo and find opportunities to deepen modules.

We're moving fast, delegating more things, and losing sense of our code base. If we lose that sense, we can't improve it — we're delegating the shape to AI.

Quality Assurance and Human Touch

- QA manually after implementation. This is where you impose your taste and opinions on the codebase.
- Automating everything (idea generation, QA, research) produces apps that lack taste or don't work. You need human touch.
- QA phase creates more issues for Kanban board. While AI implements, you QA and add blocking issues.
- Code review the tests first, then the implementation. Ensure tests are testing reasonable things.
- Once you have code you're happy with, share with team for full review before merging.

We are not producing slop here. We're trying to produce high-quality stuff, and that's what the QA is for.

],

[

The Grill Me Skill: Achieving Alignment

- Start every project with the Grill Me skill. It interviews you relentlessly about every aspect of the plan until reaching shared understanding.
- The goal is design concept alignment, not just a plan. You need to be on the same wavelength as the AI before implementation.
- AI provides recommended answers for each question. Often its recommendations are good — you can accept or modify them.
- This can take 40-100 questions and an hour of conversation. It's human-in-the-loop work that cannot be automated.
- The conversation history becomes an asset documenting the design concept. Can be fed from meeting transcripts or domain expert interviews.

We need a shared idea between all participants — that is the design concept. I needed to reach a shared understanding with Claude.

Implementation: The Ralph Loop

- Ralph Wiggum approach: specify the destination, then make small changes that get closer. Each iteration stays in smart zone.
- Use TDD (test-driven development). Write failing test first, then implement to pass. This prevents AI from cheating on tests.
- TDD adds good tests to codebase and makes it harder for AI to write implementation-first then retrofit tests.
- Feedback loops are essential. Without tests, types, and linters, AI codes blind. Quality of feedback loops determines ceiling of AI output.
- Clear context between review and implementation. Reviewing in dumb zone (after implementation tokens) produces worse results than reviewing in fresh smart zone.

If your code base doesn't have feedback loops, you're never going to get decent output from AI. The quality of your feedback loops influences how good your AI can code.

Parallelization and Automation

- Move from sequential to parallel execution. Planner chooses issues to work on in parallel, respecting blocking relationships.
- Each issue gets its own sandbox (Git branch, Docker container). Implementer runs in sandbox, creates commits, then reviewer checks work.
- Merger agent combines branches, resolves conflicts, ensures types and tests pass.
- Push coding standards to reviewer (always present), allow implementer to pull them (available when needed).
- Use different models for different roles: Sonnet for implementation, Opus for review (need the smarts for review).

Practical Tips and Tools

- Monitor token usage constantly. Essential to know how close you are to dumb zone. Add token counter to your status line.
- Use dedicated tools over terminal commands when available (file reading, editing, searching). Reserve terminal for operations that genuinely require it.
- Own your planning stack. Don't over-rely on frameworks you don't understand. When things break, you need to know how to fix them.
- Pair programming with AI works well with three people: you, domain expert, and AI. AI relentlessly quizzes you.
- Read old software engineering books (Martin Fowler, Pragmatic Programmer, Philosophy of Software Design). Pre-AI wisdom applies directly to AI development.

If you take one thing away from today, just try running the improve code base architecture skill on your repo and see what happens.

]